SÉANCE 1 SUR 3

PROGRAMMATION ORIENTÉE OBJET — JAVA

Fondamentaux Java, objets, UML et polymorphisme

Une séance progressive : syntaxe essentielle, classes, objets, UML, puis première hiérarchie polymorphe.

À LA FIN DE CETTE SÉANCE

Objectifs pédagogiques

À la fin de cette séance, vous serez capable de…

1. Lire et écrire un programme Java simple avec la syntaxe essentielle (types, structures de contrôle, tableaux)
2. Expliquer avec vos propres mots la différence entre une classe et un objet.
3. Écrire une classe avec des fields private, un constructeur et des méthodes d'accès public (encapsulation).
4. Lire un diagramme de classes UML simple et le traduire en squelette de classe Java — et inversement.
5. Construire une première hiérarchie avec extends, redéfinir une méthode et expliquer le polymorphisme.

PROGRAMME DE LA SÉANCE

A Syntaxe Java essentielle — aide-mémoire à relire avant le cours, sans présentation ligne par ligne

B Objets et classes — le cœur de la séance, avec live coding et mise en pratique

C UML, héritage et polymorphisme — représenter puis exécuter une première hiérarchie

Q1.1

Un programme Java simple

- Java est sensible à la casse et tout programme est défini dans une classe.
- Le point d'entrée de toute application est la méthode main :
  - **public static void main(String[] args)**
- La compilation et l'exécution sont deux étapes distinctes :
  - javac HelloWorld.java → produit HelloWorld.class (bytecode)
  - java HelloWorld → exécute le bytecode sur la machine virtuelle Java (JVM)
- La JVM rend Java portable : un même fichier .class s'exécute sans modification sous Windows, macOS ou Linux.

Un programme Java simple — Code

```
public class HelloWorld {

    public static void main(String[] args) {
        // point d'entrée de l'application
        System.out.println("Hello, World!");
    }

}
```

- Le nom du fichier doit correspondre au nom de la classe public : HelloWorld.java.
- // commence un commentaire sur une ligne.
- /* ... */ couvre plusieurs lignes.
- /** ... */ est un commentaire de documentation Javadoc — nous y reviendrons.

Types primitifs et constantes

Java est fortement typé : chaque variable possède un type déclaré, fixe pendant toute sa durée de vie.

Catégorie	Types	Remarques
Entiers	byte, short, int, long	int est le choix par défaut ; long pour les très grandes valeurs
Nombres à virgule flottante	float, double	double est le choix par défaut pour les décimaux
Caractère	char	un seul caractère Unicode 16 bits
Boolean	boolean	uniquement true ou false — pas de substitution par 0/1
Constante	final TYPE NAME = value;	convention : noms des constantes en MAJUSCULES

Opérateurs, String et tableaux

Q1.2 · Q1.3

Opérateurs et String

- Arithmétiques : + − * / %
- Relationnels : == != < > <= >=
- Logiques : && || !
- Affectations composées : += -= *= /=
- Les String sont immuables : chaque « modification » crée un nouvel objet String.
- Utilisez StringBuilder pour des concaténations répétées dans une boucle.

Q1.4

Tableaux

- Taille fixe après création : int[] a = new int[5];
- Indices à partir de zéro : a[0] … a[a.length-1].
- Peuvent être multidimensionnels : int[][] grid.
- Besoin d'une structure redimensionnable ? ⟶ ArrayList (aperçu en séance 2, étude complète en séance 4).
- Grands nombres dépassant la précision des primitifs : BigInteger et BigDecimal (java.math).

Entrées/sorties et structures de contrôle

Entrées / Sorties

- Scanner sc = new Scanner(System.in); lit une saisie depuis la console.
- System.out.print / println / printf écrivent dans la console.
- Les streams peuvent aussi cibler des fichiers ou des sockets réseau (plus tard dans le cursus).

Structures de contrôle

- Branchements : if / else if / else, switch.
- Boucles : while, do-while, for.
- Boucle for-each : for (int x : array) { … } — le meilleur choix lorsque l'indice n'est pas nécessaire.

Choisir une structure de contrôle

Une structure de contrôle organise l’ordre d’exécution.

Utilisez if/else pour choisir selon une condition, switch pour plusieurs cas explicites et une boucle pour répéter une action.

```
if (score >= 10) {
  result = "admis";
    } else {
 result = "ajourné";
        }
```

↓

```
switch (day) {
case SATURDAY, SUNDAY -> "week-end";
      default -> "semaine";
        }
```

↓

Une condition doit produire un boolean.

Parcourir un tableau sans dépasser ses limites

Un tableau possède une taille fixe et des indices de 0 à length - 1.

La boucle for utilise l'indice. La boucle for-each lit directement chaque valeur lorsqu'on n'a pas besoin de sa position.

```
int[] scores = {12, 15, 9};
for (int i = 0; i < scores.length; i++)
    System.out.println(scores[i]);
```

↓

```
for (int score : scores)
    System.out.println(score);
```

↓

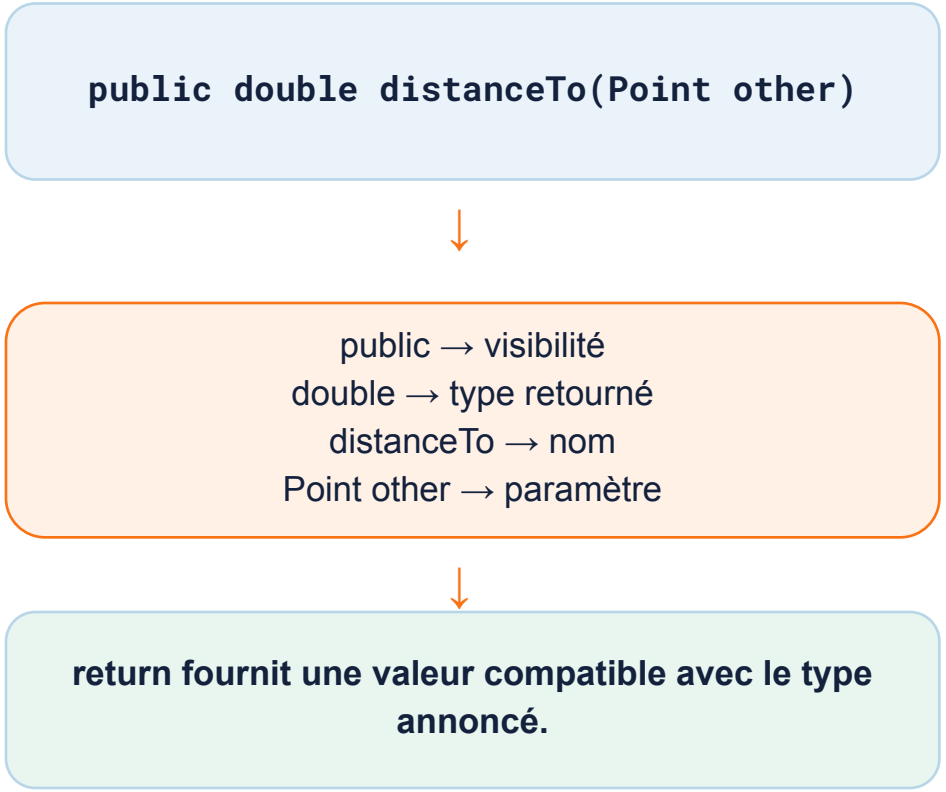
i < scores.length évite l'accès hors limites.

ANATOMIE DU CODE

Lire la signature d’une méthode

Avant d’utiliser une méthode, identifiez sa visibilité, son type de retour, son nom et ses paramètres.

Un type de retour void signifie que la méthode ne retourne aucune valeur.



LocalDate retourne une nouvelle valeur

LocalDate représente une date sans heure ni fuseau.

L’objet est immuable : plusDays ne modifie pas la date d’origine. Le programme doit conserver la référence retournée.

```
LocalDate d = LocalDate.of(2026, 10, 1);
```

```
LocalDate d2 = d.plusDays(10);
```

d reste au 1er octobre
d2 vaut le 11 octobre

Q1.5

Qu’est-ce que la programmation orientée objet ?

Approche procédurale

- Les données et les fonctions qui les manipulent sont séparées.
- Les structures de données sont transmises à des fonctions indépendantes.
- Quand le programme grandit, il devient difficile de savoir quelle fonction peut modifier quelle donnée.

Approche orientée objet

- Les données (fields) et les comportements (methods) sont regroupés dans une classe.
- Un objet n’expose que ce qu’il choisit d’exposer ; le reste est masqué.
- La conception commence par « quelles entités existent ? », puis « que peuvent-elles faire ? ».

Classe ou objet

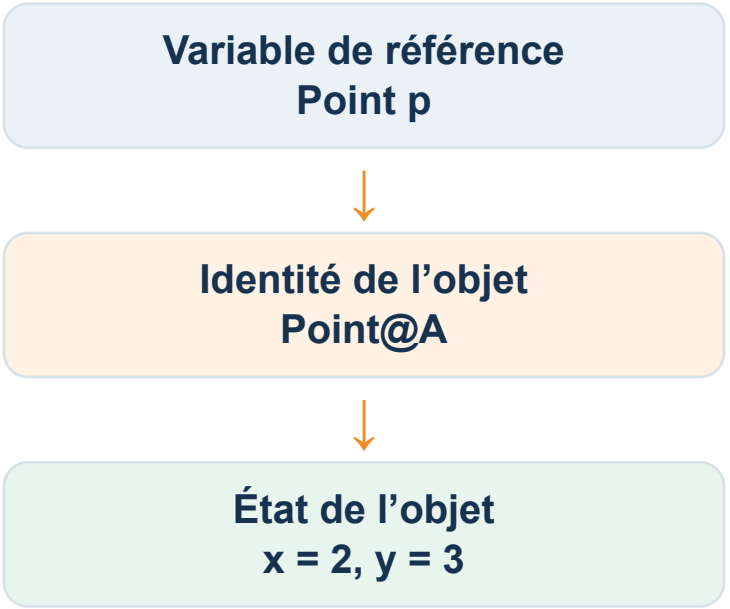
- Classe : un modèle qui définit un nouveau type, ainsi que les fields et methods de tous ses objets.
- Objet : une instance concrète construite à partir de cette classe.
- Tout objet possède un état (valeurs de ses fields), un comportement (ses methods) et une identité propre en mémoire, même si son état est identique à celui d'un autre objet.
- Analogie : la classe Car est le modèle ; myCar et yourCar sont deux objets construits sur ce modèle, avec des états indépendants (couleur et kilométrage différents).

Une variable contient une référence, pas l’objet lui-même

Une classe définit un type.

Un objet est une instance de ce type à l’exécution.

Une variable de référence contient le moyen d’accéder à cet objet.

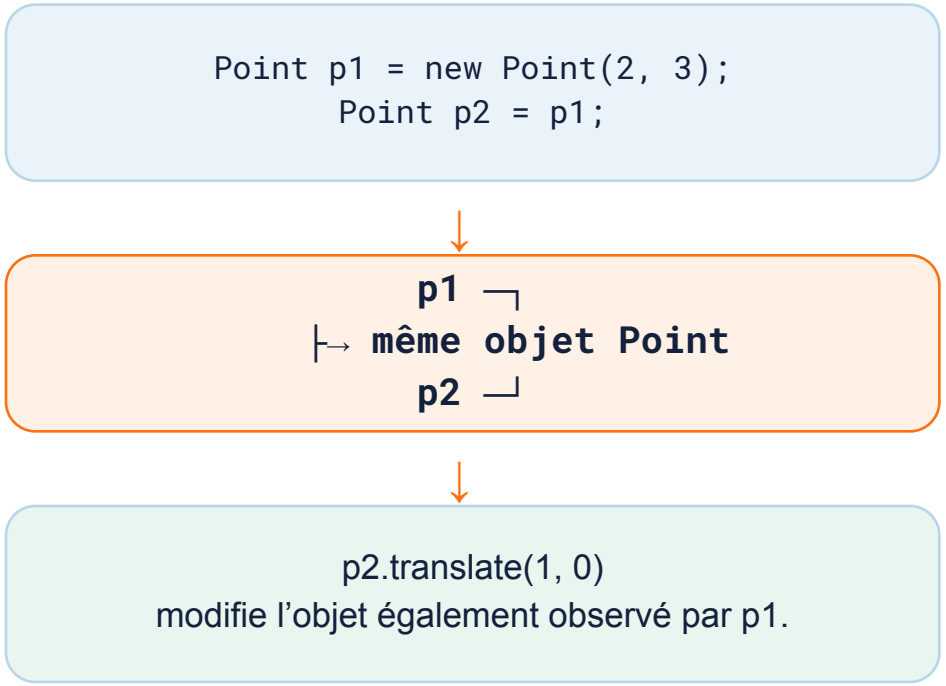


Deux variables peuvent désigner le même objet

Une variable de type classe contient une référence.

Copier cette variable ne copie pas l'objet : les deux références peuvent atteindre le même état.

null signifie « aucune référence » ; appeler une méthode sur null provoque NullPointerException.



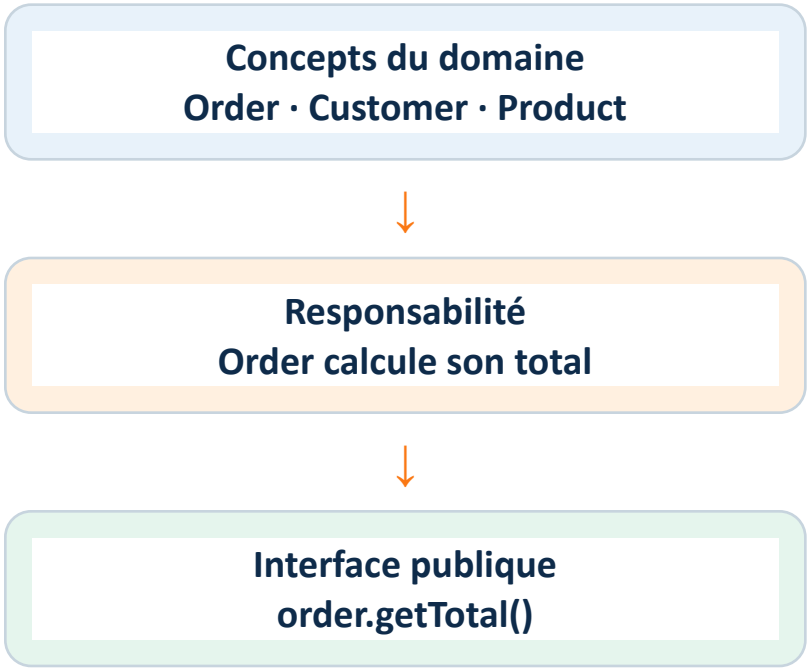
Une classe commence par une responsabilité

Avant d’écrire le code, posez trois questions :

Quels concepts existent dans le domaine ?

Quelle classe possède les données nécessaires ?

Quelle méthode doit protéger la règle métier ?



Définir votre propre classe

```
public class Point {

    private double x;
    private double y;

    public Point(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double getX() { return x; }
    public double getY() { return y; }

    public void translate(double dx, double dy) {
        this.x += dx;
        this.y += dy;
    }
}
```

- **Anatomie d'une classe :**
- Fields — l'état de l'objet
- Constructeur — création de l'objet
- Methods — comportement de l'objet
- this désigne l'objet courant ; il distingue un field d'un paramètre portant le même nom.

Encapsulation : pourquoi masquer les fields ?

```
// Fragile : rien n'empêche un état invalide
public class BadAccount {
    public int balance;
}
BadAccount a = new BadAccount();
a.balance = -500;   // aucune validation ne peut l'empêcher

// Encapsulé : la classe protège ses propres règles
public class Account {
    private int balance;

    public void deposit(int amount) {
        if (amount > 0) balance += amount;
    }
    public int getBalance() { return balance; }
}
```

- **Encapsulation = fields private + méthodes public qui font respecter les règles.**
- L'objet garantit sa propre cohérence : le code extérieur ne peut pas invalider son état.
- C'est précisément le type de validation que vous devrez implémenter dans le TD1.

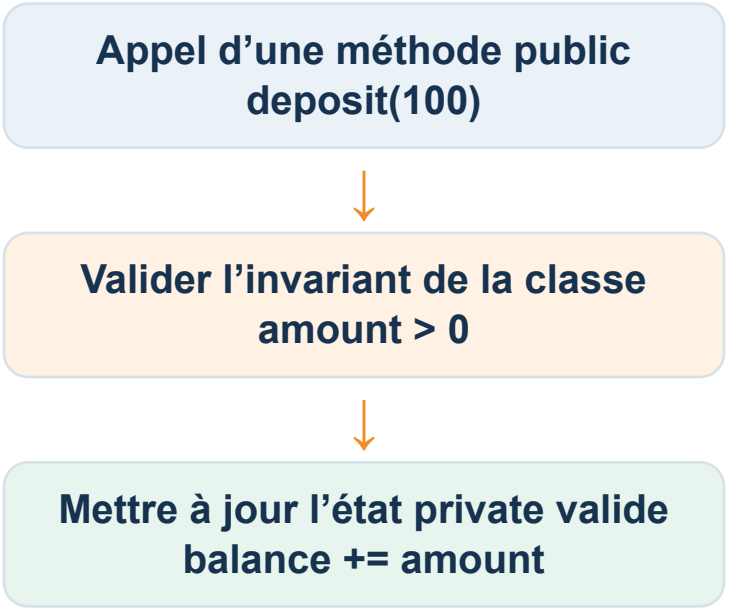
Q1.6

L’encapsulation protège la validité de l’état

Les fields private empêchent toute modification directe depuis l’extérieur.

Les méthodes public constituent la frontière contrôlée de l’objet.

Une méthode doit refuser toute modification qui violerait l’invariant de la classe.



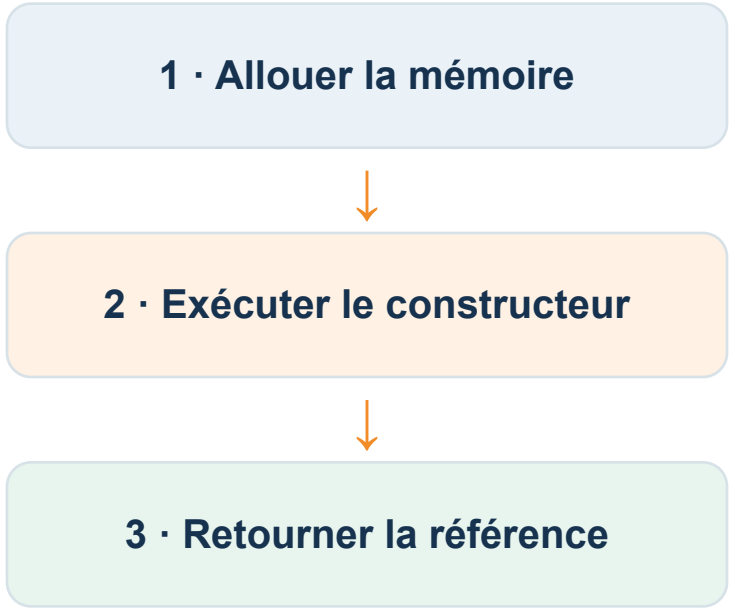
Q1.7

L’expression new crée l’identité et initialise l’état

La création d’un objet comporte trois étapes :

1. Allouer la mémoire.
2. Exécuter le constructeur.
3. Retourner une référence.

Distinguer ces étapes évite de nombreuses erreurs de débutant.



Constructeurs et overloading

```
public class Point {
    private double x, y;

    public Point(double x, double y) {
        this.x = x;
        this.y = y;
    }

    // constructeur overloaded : origine par défaut
    public Point() {
        this(0, 0);   // appelle le constructeur précédent
    }
}
```

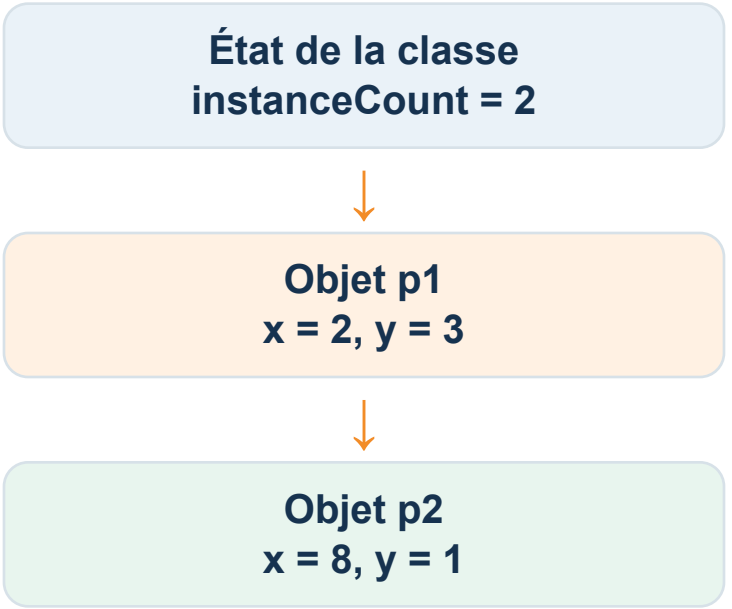
- Une classe peut avoir plusieurs constructeurs si leurs listes de paramètres sont différentes.
- this(...) en première ligne délègue à un autre constructeur de la même classe et évite de répéter l'initialisation.
- Si aucun constructeur n'est écrit, Java fournit un constructeur par défaut sans argument. Dès qu'un constructeur est déclaré, ce constructeur par défaut disparaît.

L’état d’instance et l’état static existent à des niveaux différents

Chaque objet possède ses propres fields d’instance.

Un field static possède une seule copie partagée par toute la classe.

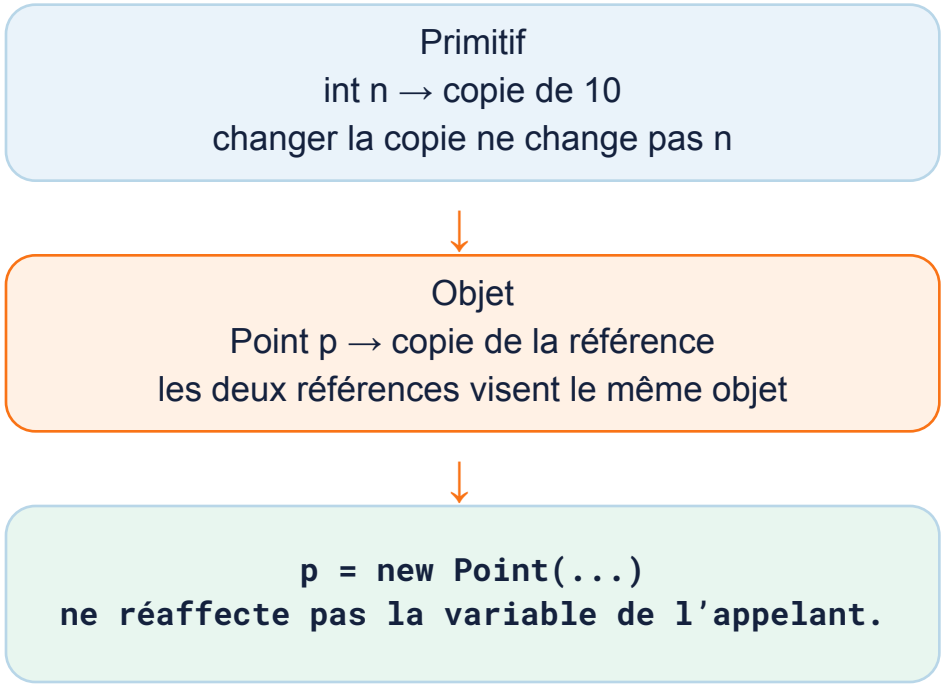
Utilisez static pour une information ou un comportement qui n’appartient pas à un objet particulier.



Java transmet toujours les arguments par valeur

Pour un primitif, la valeur est copiée.

Pour un objet, la référence est copiée : la méthode peut modifier l'objet, mais réaffecter son paramètre ne change pas la variable de l'appelant.

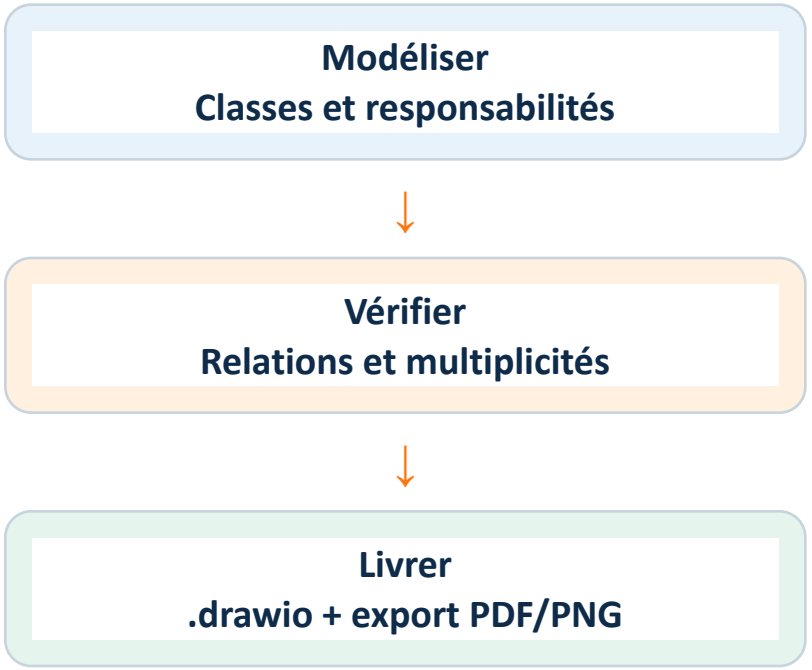


Draw.io est l’outil UML recommandé

Un diagramme évalué doit être technique, lisible et modifiable.

Utilisez la bibliothèque UML, les connecteurs normalisés et les multiplicités.

Conservez toujours le fichier source .drawio.



Draw.io · UML class diagrams

Anatomie d’une classe UML

Trois compartiments

Nom de la classe · attributs · méthodes.

Symboles de visibilité

`+` `public`

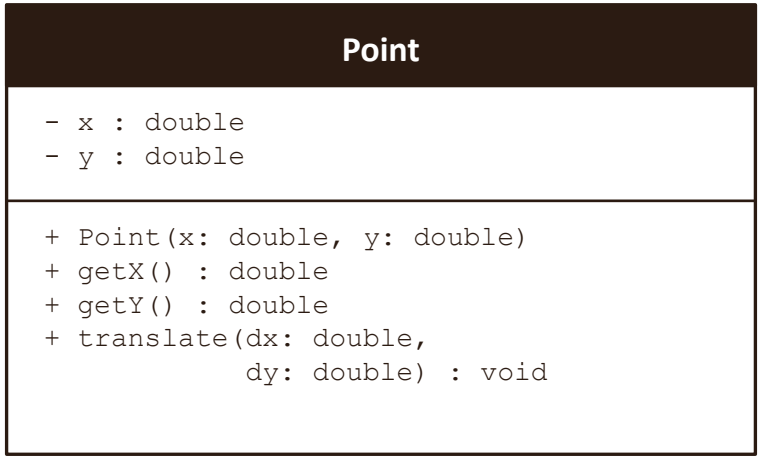
`-` `private`

`#` `protected`

Format de lecture

nom : Type pour les attributs

nom(params) : TypeRetour pour les méthodes



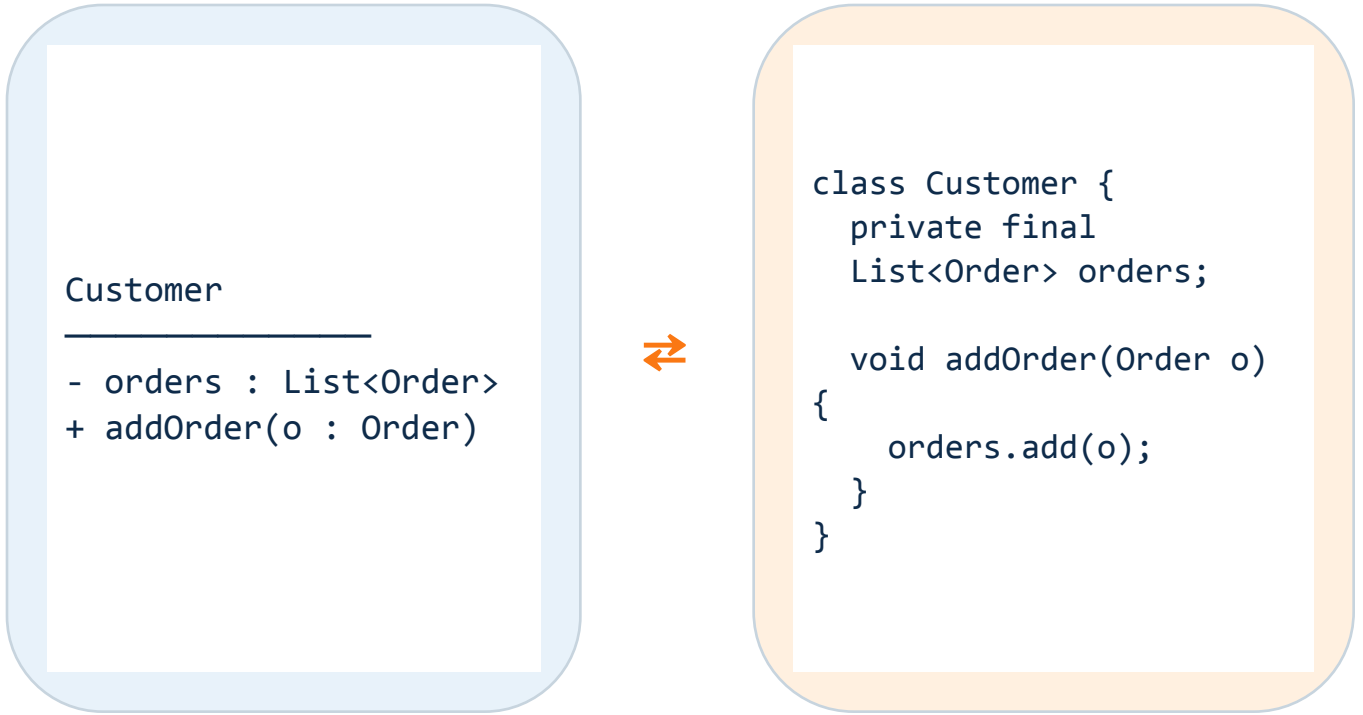
Comparez avec la classe Point écrite en Java : mêmes fields, même constructeur et mêmes methods, mais une notation différente.

La multiplicité détermine la structure du code

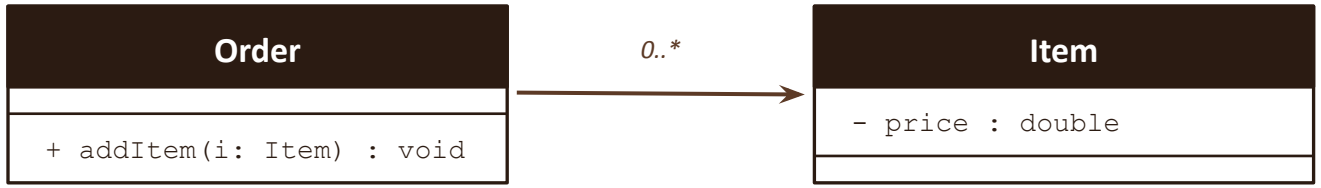
Une association durable devient généralement un champ.

La multiplicité 1 correspond à une référence.

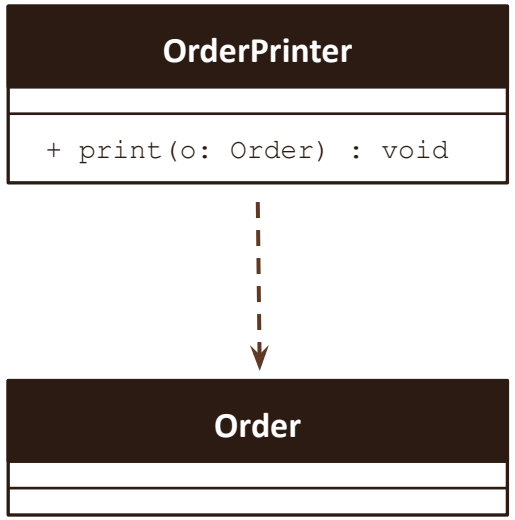
La multiplicité 0..* correspond généralement à une collection typée.



Relations (1/3) — Association et dépendance

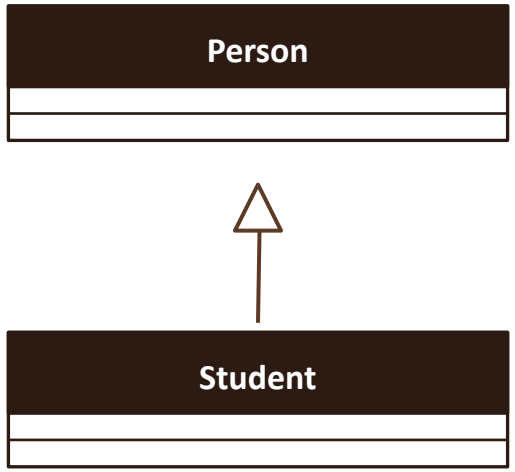


Association — lien structurel durable : Order conserve une référence vers ses Items.

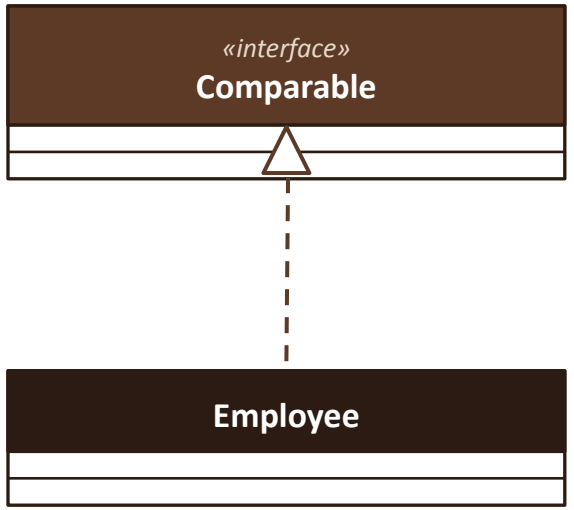


Dépendance (ligne pointillée) — utilisation temporaire par un paramètre ou une variable locale, sans field durable.

Relations (3/3) — Héritage et réalisation



Héritage (triangle creux)
Student IS-A Person. Ligne continue.

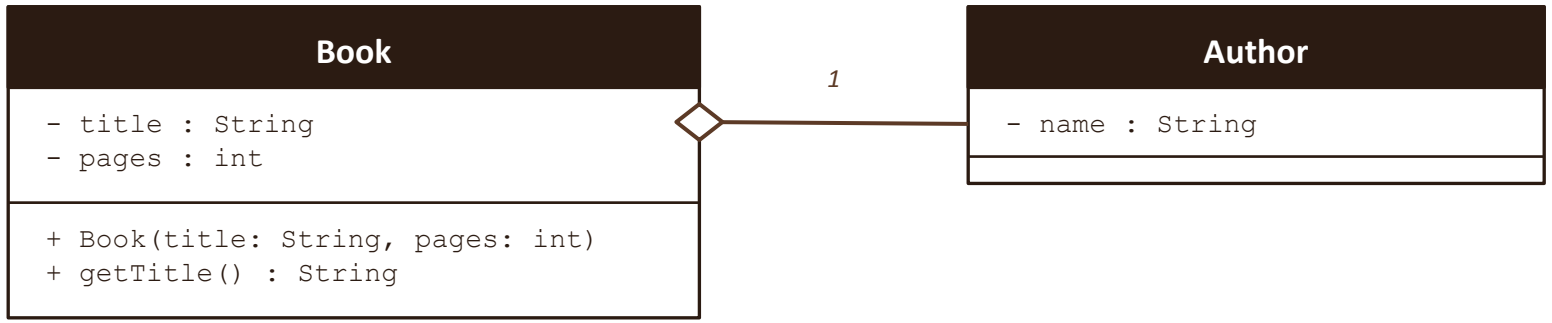


Réalisation (triangle creux, ligne pointillée)
Employee respecte le contrat Comparable. Les interfaces seront introduites en séance 3.

Multiplicités

- La multiplicité indique combien d’objets participent à une relation ; elle est écrite près de chaque extrémité.
  - 1 — exactement un
  - 0..1 — zéro ou un (facultatif)
  - 0..* (ou *) — zéro ou plusieurs
  - 1..* — un ou plusieurs
- Exemple : un Order possède 1..* Items et chaque Item appartient exactement à 1 Order.

Exemple guidé : diagramme → code



Un Book est écrit par exactement un Author : il s'agit d'une association ; utilisez l'agrégation uniquement si le cycle de vie traduit volontairement une propriété faible.

```
public class Book {
    private String title;
    private int pages;
    private Author author;   // 1 Author, issu de l'agrégation

    public Book(String title, int pages, Author author) {
        this.title = title;
        this.pages = pages;
        this.author = author;
    }
    public String getTitle() { return title; }
}
```

Ordre de lecture :

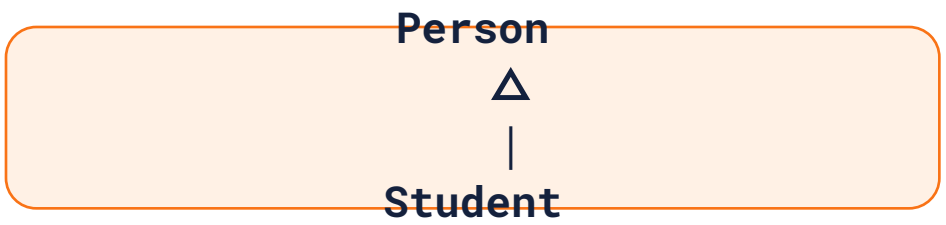
1. Chaque boîte de classe → une classe Java.
2. Chaque attribut → un field private.
3. Une relation → un field contenant une référence, ou une collection si la multiplicité vaut *.
4. Chaque signature de méthode → un squelette de méthode.

Une subclass spécialise une superclass

L’héritage représente une relation IS-A. Student est une Person et peut être utilisé partout où une Person est attendue.

La subclass reçoit les membres accessibles de la superclass et ajoute ou redéfinit son propre comportement.

```
class Student extends Person {
    private String major;
}
```



extends exprime une spécialisation, pas une simple réutilisation de code.

La superclass est initialisée avant la subclass

Un objet Student contient aussi l'état défini par Person. Le constructeur de Person doit donc s'exécuter avant celui de Student.

super(...) doit être la première instruction du constructeur.

```
Student(String name, String major) {
        super(name);
    this.major = major;
        }
```

1. Person initialise name
2. Student initialise major

Si super(...) manque, Java tente super() sans argument.

Overriding et overloading

L’overriding remplace une méthode héritée avec la même signature. Le choix de l’implémentation a lieu à l’exécution.

L’overloading conserve le même nom avec des paramètres différents. Le compilateur choisit la méthode.

```
@Override
public String describe() {
    return "Student";
}
```

describe() redéfini
Overriding

print(int) / print(String)
Overloading

Le type réel détermine la méthode exécutée

Une référence de type Person peut désigner différents objets concrets.

Le compilateur vérifie les méthodes à partir du type de la référence. À l'exécution, Java choisit la redéfinition correspondant à l'objet réel.

```
Person p = new Student("Ana", "CS");
p.describe();
```

Type de référence
Person

Objet réel
Student
Student.describe()

CONCEPTION

Héritage ou composition

L’héritage convient seulement lorsque chaque objet de la subclass peut remplacer un objet de la superclass sans rompre son contrat.

Pour une relation HAS-A, un field constitue généralement une solution plus souple.

ElectricCar est une Car
Héritage possible

Car possède un Engine
Composition

La relation métier guide le choix.

SYNTHÈSE

À retenir

- La syntaxe Java (types, structures de contrôle, tableaux) est un outil : utilisez la partie A comme référence.
- Une classe est un modèle ; un objet est une instance concrète avec son propre état et sa propre identité.
- L'encapsulation signifie que l'objet protège sa cohérence : fields private et méthodes public.
- Les constructeurs peuvent être overloaded ; les membres static appartiennent à la classe et non à une instance.
- Les diagrammes de classes UML décrivent la structure sans code : boîtes pour les classes et flèches pour les relations (association, agrégation, composition, héritage, réalisation).